240840116034

Practical - 03

Aim : Apply filtering , sorting , aggregation and GroupBy operations on real-world dataset.

```
# import pandas library
import pandas as pd
```

```
#Create a dataset.
data = {
    "Emp_ID": [101, 102, 103, 104, 105, 106, 107, 108],
    "Name": ["Rahul", "Priya", "Amit", "Riya", "Karan", "Neha", "Vivek", "Pooja"],
    "Department": ["IT", "HR","IT", "Finance", "HR", "Finance", "IT", "HR"],
    "City": ["Surat", "Navsari", "Surat", "Valsad", "Surat", "Navsari", "Valsad", "Surat"],
    "Salary": [45000, 38000, 52000, 48000, 40000, 51000, 60000, 42000],
    "Experience": [2, 1, 5, 4, 2, 6, 8,3]
    }
```

```
# convert data into table and display
df = pd.DataFrame(data)

df
```

	Emp_ID	Name	Department	City	Salary	Experience
0	101	Rahul	IT	Surat	45000	2
1	102	Priya	HR	Navsari	38000	1
2	103	Amit	IT	Surat	52000	5
3	104	Riya	Finance	Valsad	48000	4
4	105	Karan	HR	Surat	40000	2
5	106	Neha	Finance	Navsari	51000	6
6	107	Vivek	IT	Valsad	60000	8
7	108	Pooja	HR	Surat	42000	3

step 1 : Display the dataset

```
# Display the complete dtaFrame.

df
```

	Emp_ID	Name	Department	City	Salary	Experience
0	101	Rahul	IT	Surat	45000	2
1	102	Priya	HR	Navsari	38000	1
2	103	Amit	IT	Surat	52000	5
3	104	Riya	Finance	Valsad	48000	4
4	105	Karan	HR	Surat	40000	2
5	106	Neha	Finance	Navsari	51000	6
6	107	Vivek	IT	Valsad	60000	8
7	108	Pooja	HR	Surat	42000	3

step 2 : Filter data

Display Emplyoees from IT department.

```
# display rows where Department is IT.

df[df["Department"]=="IT"]
```

	Emp_ID	Name	Department	City	Salary	Experience
0	101	Rahul	IT	Surat	45000	2
2	103	Amit	IT	Surat	52000	5
6	107	Vivek	IT	Valsad	60000	8

Display emplyoees with salary greater than 50000

```
# display rows where Salary is over 50,000

df[df["Salary"]>50000]
```

	Emp_ID	Name	Department	City	Salary	Experience
2	103	Amit	IT	Surat	52000	5
5	106	Neha	Finance	Navsari	51000	6
6	107	Vivek	IT	Valsad	60000	8

Display Employees from surat

```
# display rows where City is Surat.
df[df["City"]=="Surat"]
```

	Emp_ID	Name	Department	City	Salary	Experience
0	101	Rahul	IT	Surat	45000	2
2	103	Amit	IT	Surat	52000	5
4	105	Karan	HR	Surat	40000	2
7	108	Pooja	HR	Surat	42000	3

Apply Multiple Conditions

```
# display rows where Department is IT and Salary is over 50,000.
df[(df["Department"]=="IT") & (df["Salary"]>50000)]
```

	Emp_ID	Name	Department	City	Salary	Experience
2	103	Amit	IT	Surat	52000	5
6	107	Vivek	IT	Valsad	60000	8

step 3 : sort data

Sort salary in ascending order

```
# Sort the rows in ascending order by Salary.
df.sort_values(by="Salary")
```

	Emp_ID	Name	Department	City	Salary	Experience
1	102	Priya	HR	Navsari	38000	1
4	105	Karan	HR	Surat	40000	2
7	108	Pooja	HR	Surat	42000	3
0	101	Rahul	IT	Surat	45000	2
3	104	Riya	Finance	Valsad	48000	4
5	106	Neha	Finance	Navsari	51000	6
2	103	Amit	IT	Surat	52000	5
6	107	Vivek	IT	Valsad	60000	8

Sort salary in Descending order

```
# Sort the rows in descending order by Salary.
df.sort_values(by="Salary", ascending=False)
```

	Emp_ID	Name	Department	City	Salary	Experience
6	107	Vivek	IT	Valsad	60000	8
2	103	Amit	IT	Surat	52000	5
5	106	Neha	Finance	Navsari	51000	6
3	104	Riya	Finance	Valsad	48000	4
0	101	Rahul	IT	Surat	45000	2
7	108	Pooja	HR	Surat	42000	3
4	105	Karan	HR	Surat	40000	2
1	102	Priya	HR	Navsari	38000	1

sort by department and salary

```
# Sort by Department (A-Z), then by Salary highest to lowest.

df.sort_values(by=["Department", "Salary"], ascending=[True, False])
```

	Emp_ID	Name	Department	City	Salary	Experience
5	106	Neha	Finance	Navsari	51000	6
3	104	Riya	Finance	Valsad	48000	4
7	108	Pooja	HR	Surat	42000	3
4	105	Karan	HR	Surat	40000	2
1	102	Priya	HR	Navsari	38000	1
6	107	Vivek	IT	Valsad	60000	8
2	103	Amit	IT	Surat	52000	5
0	101	Rahul	IT	Surat	45000	2

step 4 : perform aggregation

Calculate Total salary

```
# Calculate the total salary of all employees.
df["Salary"].sum()
```

```
np.int64(376000)
```

Calculate Average Salary

```
# Calculate the average salary.
df["Salary"].mean()
```

```
np.float64(47000.0)
```

Find Highest Salary

```
# dilspay the maximum salary.
df["Salary"].max()
```

```
60000
```

Find lowest salary

```
# Diplay the minimum salary.
df["Salary"].min()
```

```
38000
```

Count emplyoees

```
# Count the total number of emplyoees.
df["Emp_ID"].count()
```

```
np.int64(8)
```

step 5 : Perform GroupBy Operations

Average salary department-wise

```
# Calculate the average salary for each department

df.groupby("Department")["Salary"].mean()
```

Department	Salary
Finance	49500.000000
HR	40000.000000
IT	52333.333333

dtype: float64

Minimum salary department-wise

```
# Find the minimum salary in each department.
df.groupby("Department")["Salary"].min()
```

Department	Salary
Finance	48000
HR	38000
IT	45000

dtype: int64

Count salary department-wise

```
# Count the number of employees (salaries) in each department.
df.groupby("Department")["Salary"].count()
```

Department	Salary
Finance	2
HR	3
IT	3

dtype: int64

Total salary department-wise

```
# Calculate the total salary for each department

df.groupby("Department")["Salary"].sum()
```

Department	Salary
Finance	99000
HR	120000
IT	157000

dtype: int64

Maximum Salary Department-wise

```
# Find the maximum salary in each department

df.groupby("Department")["Salary"].max()
```

Department	Salary
Finance	51000
HR	42000
IT	60000

dtype: int64

Average salary department-wise

```
# Calculate the average salary for each department.

df.groupby("Department")["Salary"].mean()
```

Department	Salary
Finance	49500.000000
HR	40000.000000
IT	52333.333333

dtype: float64

Group by city

```
# calculate the average salary of each city.
df.groupby("City")["Salary"].mean()
```

City	Salary
Navsari	44500.0
Surat	44750.0
Valsad	54000.0

dtype: float64

step 6 : Multiple Aggregation Functions

```
# Calculate multiple statistical functions to the slary column for each department.

df.groupby("Department")["Salary"].agg(["count", "sum", "mean", "min", "max"])
```

Department	count	sum	mean	min	max
Finance	2	99000	49500.000000	48000	51000
HR	3	120000	40000.000000	38000	42000
IT	3	157000	52333.333333	45000	60000

step 7 : save the result

```
# Save DataFrame to CSV without index numbers

df.to_csv("employee_data.csv", index=False)
print("Dataset saved successfully.")
```

```
Dataset saved successfully.
```

Dowmload the file

```
# save the DataFrame as a CSV file
df.to_csv("employee_data.csv", index=False)

# Import the files module from google colab.
from google.colab import files

# Download the CSV file
files.download("employee_data.csv")
```